

# Human Vision – Topic 4

## The Retinex Algorithm

Yonghao Lee

May 10, 2026

### Contents

|          |                                                            |          |
|----------|------------------------------------------------------------|----------|
| <b>1</b> | <b>Recap: The Inverse Optics Problem</b>                   | <b>2</b> |
| 1.1      | Intrinsic Images (Barrow and Tenenbaum, 1978) . . . . .    | 2        |
| 1.2      | The Fundamental Ambiguity . . . . .                        | 3        |
| <b>2</b> | <b>The Retinex Algorithm</b>                               | <b>3</b> |
| 2.1      | Core Assumption: Sharp vs. Soft Gradients . . . . .        | 3        |
| 2.2      | The Algorithm in Log Space . . . . .                       | 3        |
| 2.3      | Retinex in 1D . . . . .                                    | 4        |
| 2.4      | Retinex in 2D . . . . .                                    | 5        |
| 2.5      | Recovering an Image from its Derivatives . . . . .         | 6        |
| <b>3</b> | <b>Does Human Vision Use Retinex?</b>                      | <b>7</b> |
| 3.1      | Leonardo da Vinci on Shadows: Umbra and Penumbra . . . . . | 7        |
| 3.2      | Two Examples in Art . . . . .                              | 8        |
| 3.3      | The Penumbra Test (A Fun Thing to Do) . . . . .            | 8        |
| <b>4</b> | <b>Summary</b>                                             | <b>9</b> |

---

# 1 Recap: The Inverse Optics Problem

In the previous topic we built up the *inverse optics* problem. Under Lambertian reflectance, the light reaching the eye is

$$I = \rho L \cos(\theta),$$

but what we actually care about is  $\rho$ , the albedo – the underlying “paint color” that tells us whether someone is wearing a white shirt or a black one. The eye only ever sees the product, and one equation with three unknowns is mathematically hopeless. Adelson’s Checker-Shadow illusion convinced us that the brain solves this anyway, by leaning on its understanding of the lighting in the scene.

In this topic we move from *why* the brain has to do this to *how* it might. We look at a concrete computational algorithm – the **Retinex algorithm**, originally proposed by Edwin Land – that tries to do something similar from a single image. It is one of the earliest serious attempts to decompose an image into its underlying physical causes, and it is the simplest example of an idea that still drives a lot of vision research today.

## 1.1 Intrinsic Images (Barrow and Tenenbaum, 1978)

Before we can describe the algorithm we need a cleaner statement of the goal. Barrow and Tenenbaum proposed that the visual system computes an intermediate, midlevel representation of a scene called **intrinsic images**: instead of producing a single output image, it factors the input into separate maps of reflectance and illumination.

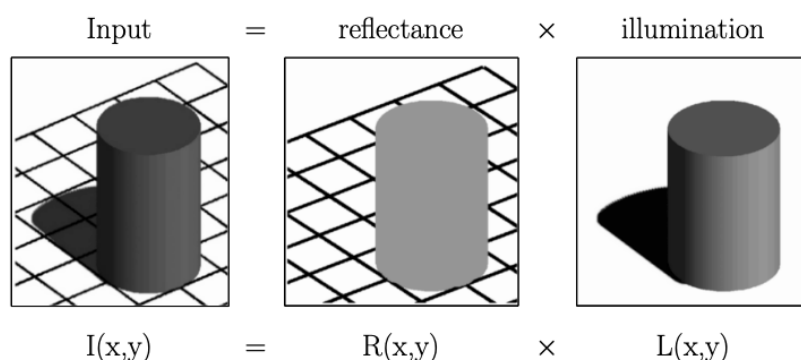


Figure 1: Intrinsic images: the input image  $I(x, y)$  is decomposed into a reflectance map  $R(x, y)$  and an illumination map  $L(x, y)$ , related by  $I = R \cdot L$ .

### Definition 1.1 Intrinsic Images

Given an input image  $I(x, y)$ , the intrinsic image decomposition is a factorization

$$I(x, y) = R(x, y) \cdot L(x, y)$$

into a **reflectance image**  $R(x, y)$  (the albedo at every point) and an **illumination image**  $L(x, y)$  (the amount of light landing on that point, including effects of surface orientation and shadows). It is a midlevel description of the scene – not the final percept, but a useful intermediate representation that the rest of the visual system can build on.

## 1.2 The Fundamental Ambiguity

The decomposition  $I = R \cdot L$  is not unique. For any pair  $(R, L)$  that explains the image,  $(\alpha R, L/\alpha)$  explains it equally well for any positive  $\alpha$ . A nice way to feel this:

### Two Numbers Multiplying to 64

“I am thinking of two numbers, their product is 64. What are the numbers?”

We cannot answer without an extra assumption – it could be  $8 \times 8$ ,  $4 \times 16$ ,  $2 \times 32$ , or any of infinitely many pairs. Recovering  $R$  and  $L$  from a single image is exactly this problem repeated at every pixel. The Retinex algorithm works by committing to one specific assumption about how reflectance and illumination differ from each other.

## 2 The Retinex Algorithm

### 2.1 Core Assumption: Sharp vs. Soft Gradients

The Retinex algorithm rests on one strong, almost reckless assumption about the statistics of natural scenes:

#### The Retinex Assumption

Every gradient (intensity change) in an image is caused either by a change in reflectance or by a change in illumination – never both at once. Furthermore:

- **Sharp gradients** (abrupt jumps in intensity) are due to **reflectance** changes – e.g., the boundary between a black tile and a white tile.
- **Soft gradients** (gradual changes in intensity) are due to **illumination** changes – e.g., the slow falloff of light across a wall, or the soft edge of a shadow.

#### Assumptions are just assumptions

This is not a law of nature. Some real reflectance changes are gradual (a smoothly painted gradient), and some real illumination changes are sharp (the hard edge of a focused spotlight). In those cases the algorithm will simply be wrong. But the assumption is statistically reasonable for *most* natural scenes, and that is enough to get a working algorithm out of it.

### 2.2 The Algorithm in Log Space

The first trick is to take logarithms, which turns the awkward product into a sum:

$$I(x, y) = L(x, y) R(x, y)$$

$$\log I(x, y) = \log L(x, y) + \log R(x, y).$$

Using lowercase letters for the log-images:

$$i(x, y) = l(x, y) + r(x, y).$$

Now reflectance and illumination contribute additively, and – more importantly for us – so do their gradients:

$$\nabla i = \nabla l + \nabla r.$$

This is what makes the assumption operational. We have one observed quantity,  $\nabla i$ , and a clean rule for splitting it between  $\nabla l$  and  $\nabla r$ .

### The Retinex Algorithm

Given the log-image  $i(x, y)$  and a threshold  $T$ :

1. Compute the gradient  $\nabla i$  at every pixel.
2. For each pixel, classify the gradient by magnitude:
  - If  $\|\nabla i\| < T$ : assign it to illumination, set  $\nabla l = \nabla i$  and  $\nabla r = 0$ .
  - If  $\|\nabla i\| > T$ : assign it to reflectance, set  $\nabla r = \nabla i$  and  $\nabla l = 0$ .
3. Integrate the two gradient fields back to recover  $l(x, y)$  and  $r(x, y)$ .
4. Exponentiate to recover  $L$  and  $R$ .

This is a simplified version of Retinex; real implementations use more sophisticated path-based or Poisson-based variants, but the spirit is the same.

## 2.3 Retinex in 1D

The cleanest way to see what is going on is in 1D. Imagine a 1D “image”  $i(x)$  built from a sharp reflectance step  $r(x)$  added to a slowly-varying illumination ramp  $l(x)$ .

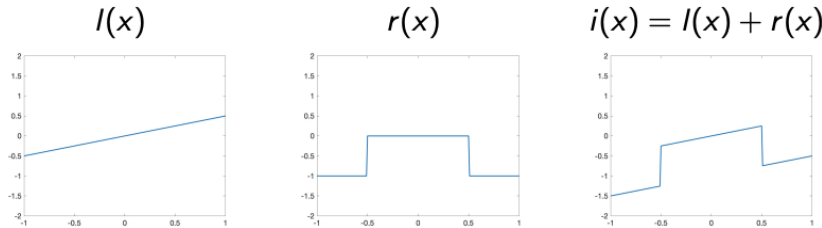


Figure 2: 1D Retinex setup. Left: a smooth illumination signal  $l(x)$ . Middle: a sharp reflectance step  $r(x)$ . Right: their sum  $i(x) = l(x) + r(x)$ , which is the only thing the algorithm receives as input.

The question is whether we can recover both  $l(x)$  and  $r(x)$  given only  $i(x)$ . With the Retinex assumption, the answer is yes, almost trivially. Differentiating  $i$  produces a single tall spike (the reflectance edge) sitting on a small, roughly constant offset (the smooth illumination ramp). Thresholding cleanly peels them apart, and then we just integrate each piece back.

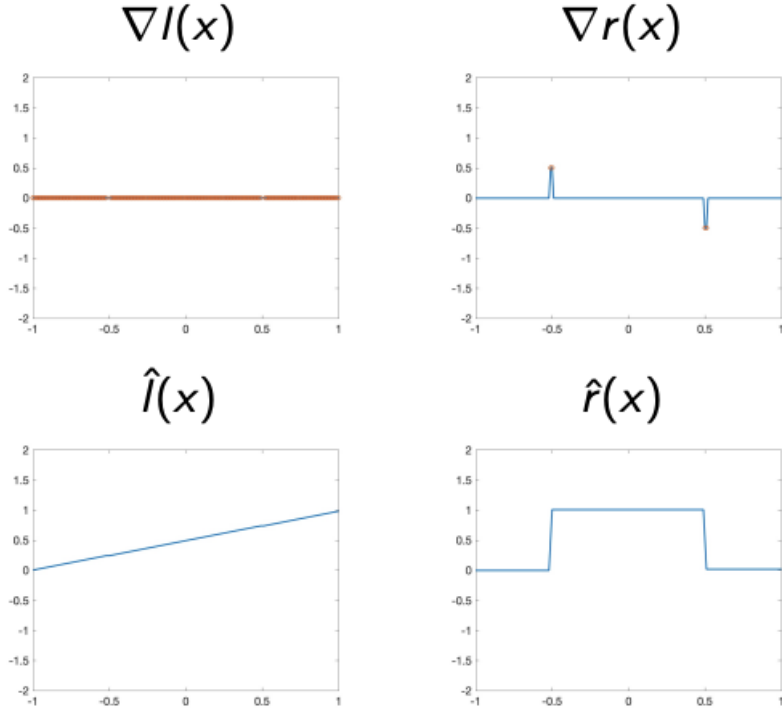


Figure 3: 1D Retinex recovery. Top: the gradient  $\nabla i(x)$  shows a tall reflectance spike sitting on the small constant offset of the illumination ramp. Middle: thresholding splits it into  $\nabla l(x)$  and  $\nabla r(x)$ . Bottom: integrating each gradient field recovers the estimates  $\hat{l}(x)$  and  $\hat{r}(x)$ .

In 1D, “integrate” just means “cumulative sum,” so the recovery is essentially exact. The one thing we cannot recover is the absolute level of brightness – only the relative differences across pixels. This will be a recurring theme.

## 2.4 Retinex in 2D

The 2D version follows exactly the same recipe:

$$i(x, y) = l(x, y) + r(x, y), \quad \nabla i = \nabla l + \nabla r.$$

- Compute  $\nabla i$ .
- Threshold:  $\|\nabla i\| < T \Rightarrow \nabla l = \nabla i$ ;  $\|\nabla i\| > T \Rightarrow \nabla r = \nabla i$ .
- Integrate  $\nabla l$  and  $\nabla r$  to recover  $l$  and  $r$ .

### Why 2D is Harder Than 1D

In 1D, integrating a derivative is trivial. In 2D it is not. After thresholding we have edited the gradient field pixel by pixel without regard to integrability, and the result is generally **not the gradient of any function**. We can no longer just “add up” the values along a path back to  $l$  or  $r$ , because the answer would depend on which path we picked. We have to settle for the function whose gradient is *closest* to the field we want.

## 2.5 Recovering an Image from its Derivatives

This is the part of the algorithm that is worth working through carefully. Suppose someone hands us the horizontal and vertical derivative images of an unknown image  $x$ :

$$u = x \star K_x, \quad v = x \star K_y,$$

where  $K_x$  and  $K_y$  are simple derivative kernels (something like  $[-1, 1]$  horizontally and vertically). Can we recover  $x$ ?

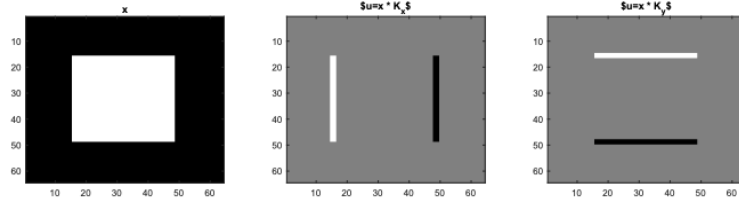


Figure 4: From derivatives back to images in 2D. Given the original  $x$  (left), we compute its horizontal derivative  $u = x \star K_x$  (middle) and vertical derivative  $v = x \star K_y$  (right). The recovery problem is to invert this process.

The natural move is to set this up as a least-squares problem: find the  $x$  whose derivatives most closely match what we observed,

$$J(x) = \|K_x \star x - u\|^2 + \|K_y \star x - v\|^2.$$

### Definition 2.1 The Discrete Laplacian

Define the Laplacian kernel

$$K_l = K_x \star K_x + K_y \star K_y.$$

### Theorem: Poisson Equation for Image Recovery

The minimizer of  $J(x)$  satisfies

$$K_l \star x = u \star K_x + v \star K_y.$$

So the optimal reconstruction is whatever solves a 2D Poisson equation, with right-hand side built from the observed derivative fields.

### Recovery up to a Constant

Suppose there exists a kernel  $K_\delta$  that inverts  $K_l$  on zero-mean signals – i.e.,  $K_\delta \star K_l \star x = x$  for any  $x$  with zero mean. Then

$$\hat{x} = K_\delta \star (u \star K_x + v \star K_y)$$

recovers  $x$  **up to an additive constant**. This is the same observation as in the 1D case, and it has a simple physical reading: derivatives are blind to constant offsets, so we can never recover the absolute brightness of an image – only the relative brightness across pixels.

This Poisson reconstruction is the engine that closes the loop in 2D. We threshold  $\nabla i$  to obtain  $\nabla l$  and  $\nabla r$ , then plug each gradient field into the Poisson equation to get back  $l$  and  $r$ .

### 3 Does Human Vision Use Retinex?

Retinex is a computational story, but the reason it shows up in a vision course is that the same prior seems to be baked into our own perception. The honest version of the question is:

#### The Better Question

Do we (humans) *assume* that soft gradients are illumination changes?

If we do, then artists and illusion-designers ought to be able to manipulate our percepts simply by playing with the softness of edges. As it turns out, they have been doing exactly that for a very long time.

#### 3.1 Leonardo da Vinci on Shadows: Umbra and Penumbra

In his notes on painting, Leonardo da Vinci described in detail how to render a convincing shadow. His key insight was that a shadow should never be drawn as a single hard-edged dark patch. It needs a darker central region – the **umbra** – surrounded by a softer transition zone – the **penumbra** – where the shadow gradually fades into the lit area.

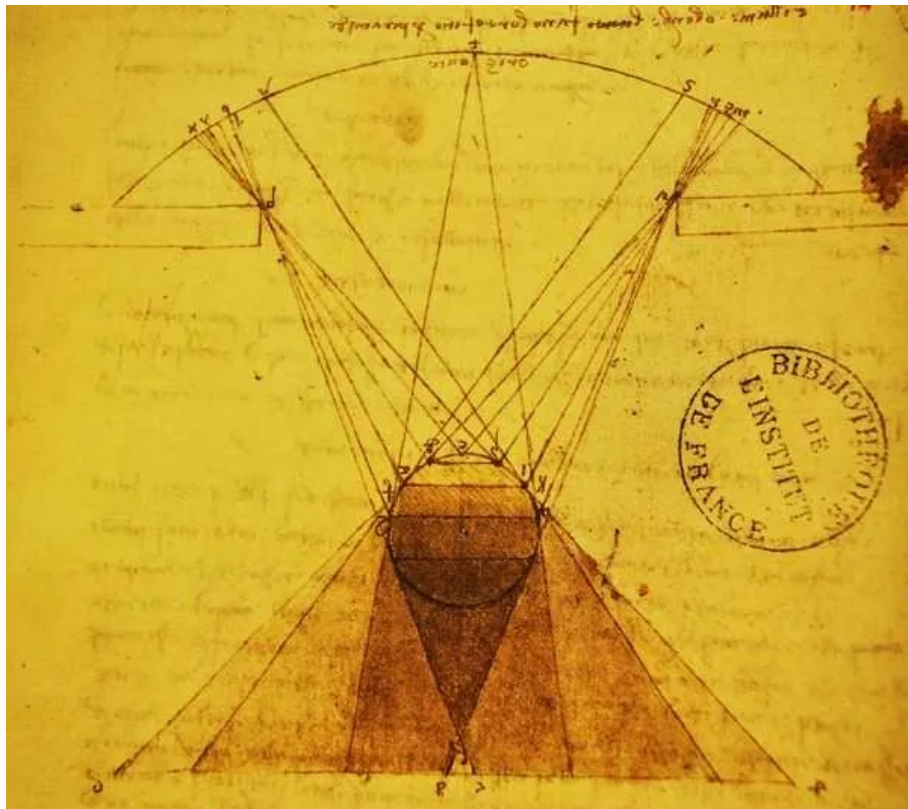


Figure 5: The structure Leonardo prescribes for a shadow: a dark central *umbra*, surrounded by a graded *penumbra* that softens the transition into the lit region.

### Definition 3.1 Umbra and Penumbra

For an extended light source partially blocked by an occluder:

- The **umbra** is the region where the light source is fully blocked. It is uniformly dark.
- The **penumbra** is the region where the light source is only partially blocked, producing a smooth gradient from dark (umbra side) to fully lit.

The penumbra is what makes the boundary of a real shadow *soft* rather than sharp.

The reason this matters for Retinex is that the penumbra is, by construction, a *soft* gradient. If our visual system uses the soft = illumination rule, then a penumbra is exactly what is needed to make the brain register the dark region as a shadow rather than as a piece of black paint.

## 3.2 Two Examples in Art

When we look at masterworks of representational painting, we find that successful renderings of cast shadows almost always include carefully painted penumbrae. Without them, the dark patch reads as a piece of paint stuck to the surface, not as a region of reduced illumination.

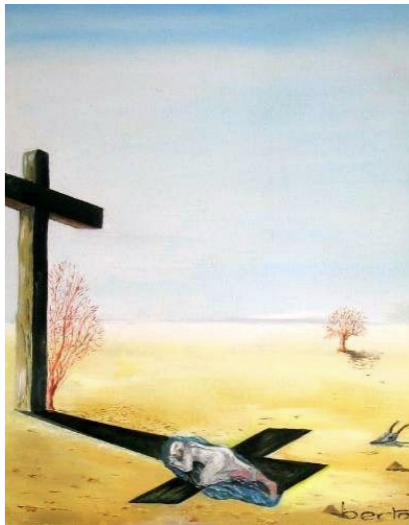


Figure 6: Two examples from art history in which the painter explicitly renders the soft penumbra around a cast shadow. The graded edge is what makes the eye interpret the dark region as a shadow rather than as a change of paint.

## 3.3 The Penumbra Test (A Fun Thing to Do)

There is a satisfying demonstration of how committed our visual system is to the soft-gradient = illumination rule. Take any image with a clear cast shadow and use a black marker to fill in the penumbra, replacing the gradual edge with an abrupt one. The luminance values inside the shadow region are essentially unchanged – only the *softness of the boundary* has been altered. Yet the perceptual interpretation flips entirely.

### Why this is a Retinex-style demonstration

The luminance values inside the shadow are essentially unchanged. Only the *softness of the boundary* has been altered. Yet the perceptual interpretation flips entirely:

- Soft edge  $\Rightarrow$  classified as illumination  $\Rightarrow$  “this is a shadow.”
- Sharp edge  $\Rightarrow$  classified as reflectance  $\Rightarrow$  “this is dark paint.”

This is exactly the Retinex thresholding rule, applied by the brain.

## 4 Summary

### Retinex in One Page

- **Goal:** factor an input image into intrinsic images of reflectance  $R(x, y)$  and illumination  $L(x, y)$ , where  $I = R \cdot L$  (Barrow and Tenenbaum, 1978).
- **Core assumption:** sharp gradients in an image are due to reflectance; soft gradients are due to illumination.
- **Method:** take logs to make the problem additive ( $i = l + r$ ), threshold  $\nabla i$  to split it into  $\nabla l$  and  $\nabla r$ , then integrate each field back to a function. In 2D the integration step is a Poisson equation built from the Laplacian.
- **Input/output:** one image in, two images out.
- **Limit:** reconstruction is only possible up to an additive constant – absolute brightness is unrecoverable.
- **Connection to perception:** the importance of the penumbra in convincingly rendered shadows – noticed by Leonardo, exploited by painters, and demonstrated by the marker test – suggests our visual system uses the same soft/sharp prior.